

Your Title Here

Name(s): (your name(s) here)

Website Link: (your website link)

```
In [1]: import pandas as pd
import numpy as np
from pathlib import Path

import plotly.express as px
import plotly.io as pio
pio.renderers.default = "notebook"
pd.options.plotting.backend = 'plotly'

# from dsc259r_utils import * # Feel free to uncomment and use this.
```

Step 1: Introduction

What features predict recipe popularity (number of ratings)?

Popularity is defined as the **number of non-zero ratings** a recipe receives in the interactions table. This focuses on engagement/visibility rather than the average star rating.

```
In [2]: # TODO
```

Step 2: Data Cleaning and Exploratory Data Analysis

```
In [3]: import pandas as pd
import numpy as np
import ast
import plotly.express as px
import plotly.io as pio

pio.renderers.default = "notebook"
pd.options.plotting.backend = "plotly"

recipes = pd.read_csv("RAW_recipes.csv")
interactions = pd.read_csv("RAW_interactions.csv")
```

```
print("recipes:", recipes.shape)
print("interactions:", interactions.shape)

recipes.head()
```

```
recipes: (83782, 12)
interactions: (731927, 5)
```

```
Out[3]:
```

	name	id	minutes	contributor_id	submitted	tags	nutrition
0	1 brownies in the world best ever	333281	40	985201	2008-10-27	['60- minutes- or-less', 'time-to- make', 'course...	[138.4, 10.0, 50.0, 3.0, 3.0, 19.0, 6.0]
1	1 in canada chocolate chip cookies	453467	45	1848091	2011-04-11	['60- minutes- or-less', 'time-to- make', 'cuisin...	[595.1, 46.0, 211.0, 22.0, 13.0, 51.0, 26.0]
2	412 broccoli casserole	306168	40	50969	2008-05-30	['60- minutes- or-less', 'time-to- make', 'course...	[194.8, 20.0, 6.0, 32.0, 22.0, 36.0, 3.0]
3	millionaire pound cake	286009	120	461724	2008-02-12	['time-to- make', 'course', 'cuisine', 'prepara...	[878.3, 63.0, 326.0, 13.0, 20.0, 123.0, 39.0]
4	2000 meatloaf	475785	90	2202916	2012-03-06	['time-to- make', 'course', 'main- ingredient', ...	[267.0, 30.0, 12.0, 12.0, 29.0, 48.0, 2.0]

```
In [4]: # --- Parse / clean recipes ---
# Parse list-like columns stored as strings
for col in ["tags", "nutrition", "ingredients", "steps"]:
    if col in recipes.columns:
        recipes[col] = recipes[col].apply(ast.literal_eval)
```

```

# Convert dates
recipes["submitted"] = pd.to_datetime(recipes["submitted"], errors="co

# Expand nutrition vector into columns (7 values)
nutrition_cols = ["calories", "total_fat", "sugar", "sodium", "protein
nutrition_df = pd.DataFrame(recipes["nutrition"].to_list(), columns=nu

recipes_clean = pd.concat([recipes.drop(columns=["nutrition"]), nutrit

# Log cook time (very skewed)
recipes_clean["log_minutes"] = np.log1p(recipes_clean["minutes"])

recipes_clean[["id", "name", "minutes", "n_steps", "n_ingredients", "s

```

Out[4]:

	id	name	minutes	n_steps	n_ingredients	submitted	calories	to
--	----	------	---------	---------	---------------	-----------	----------	----

0	333281	1 brownies in the world best ever	40	10	9	2008-10-27	138.4	
1	453467	1 in canada chocolate chip cookies	45	12	11	2011-04-11	595.1	
2	306168	412 broccoli casserole	40	6	9	2008-05-30	194.8	
3	286009	millionaire pound cake	120	7	7	2008-02-12	878.3	
4	475785	2000 meatloaf	90	17	13	2012-03-06	267.0	

In [5]:

```

# --- Parse / clean interactions ---
interactions["date"] = pd.to_datetime(interactions["date"], errors="co

# Remove placeholder/non-ratings (rating == 0)
interactions_clean = interactions[interactions["rating"] > 0].copy()

# Popularity = number of (non-zero) ratings per recipe
popularity = (
    interactions_clean
    .groupby("recipe_id")
    .size()
    .reset_index(name="num_ratings")
)

```

```
# Merge popularity back to recipe-level table
recipe_level = recipes_clean.merge(popularity, left_on="id", right_on=
recipe_level["num_ratings"] = recipe_level["num_ratings"].fillna(0).as

# Log popularity (very skewed)
recipe_level["log_num_ratings"] = np.log1p(recipe_level["num_ratings"])

recipe_level[["id", "name", "num_ratings", "log_num_ratings", "minutes
```

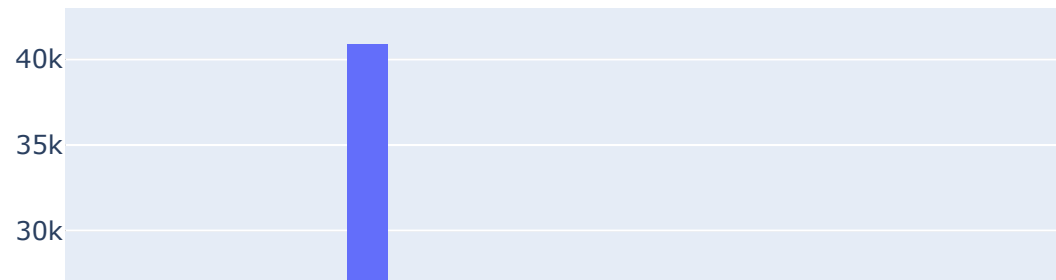
Out[5]:

	id	name	num_ratings	log_num_ratings	minutes	log_minutes	n_
0	333281	1 brownies in the world best ever	1	0.693147	40	3.713572	
1	453467	1 in canada chocolate chip cookies	1	0.693147	45	3.828641	
2	306168	412 broccoli casserole	4	1.609438	40	3.713572	
3	286009	millionaire pound cake	1	0.693147	120	4.795791	
4	475785	2000 meatloaf	2	1.098612	90	4.510860	

In [6]:

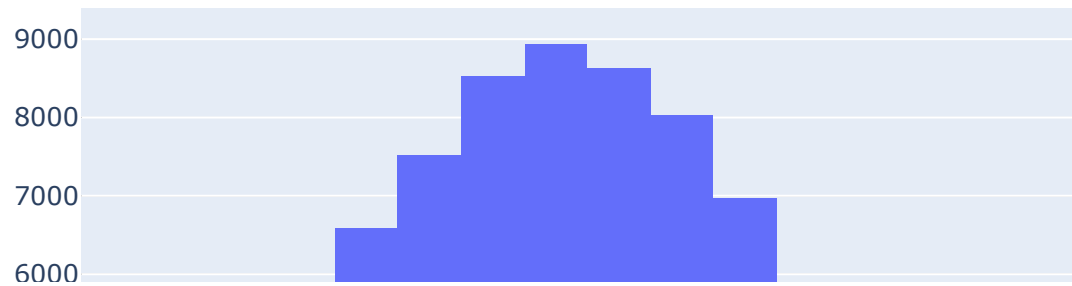
```
# Univariate: popularity distribution (log scale)
fig = px.histogram(
    recipe_level,
    x="log_num_ratings",
    nbins=60,
    title="Log Distribution of Recipe Popularity (Number of Ratings)"
)
fig.show()
```

Log Distribution of Recipe Popularity (Number of Ratings



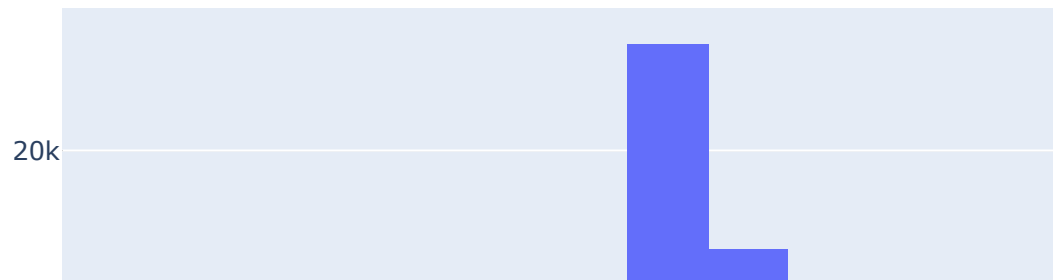
```
In [7]: # Univariate: ingredient count distribution
fig = px.histogram(
    recipe_level,
    x="n_ingredients",
    nbins=50,
    title="Distribution of Number of Ingredients"
)
fig.show()
```

Distribution of Number of Ingredients



```
In [8]: # Univariate: cook time distribution (log scale)
fig = px.histogram(
    recipe_level,
    x="log_minutes",
    nbins=60,
    title="Log Distribution of Cook Time (minutes)"
)
fig.show()
```

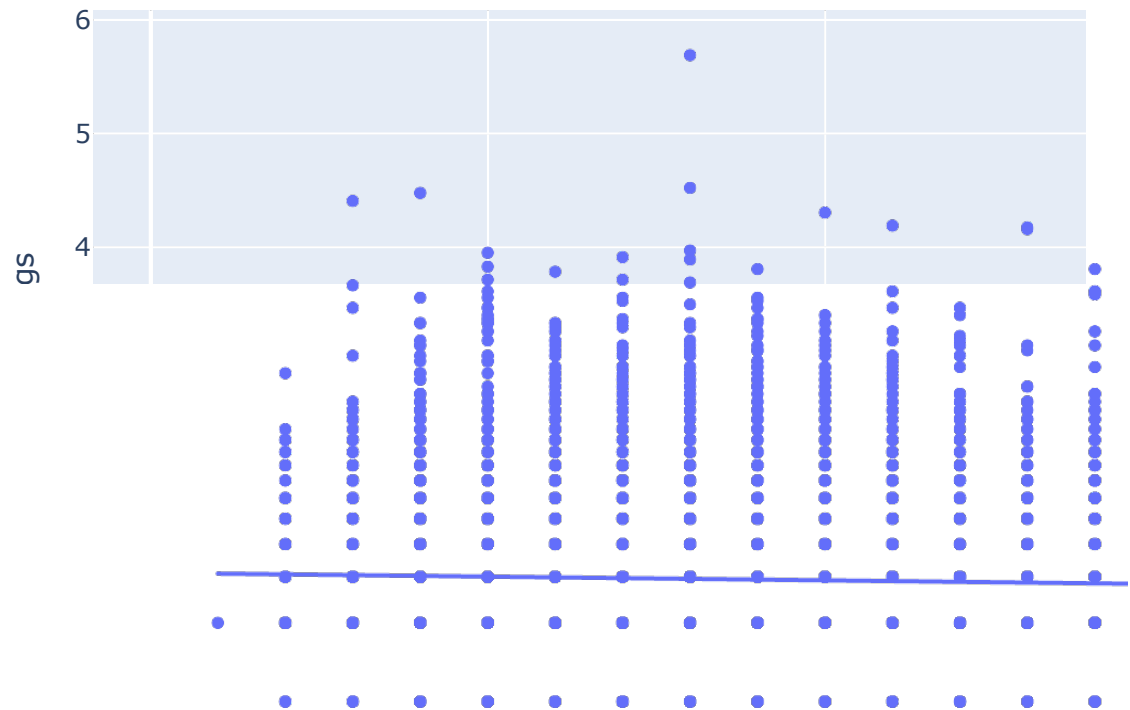
Log Distribution of Cook Time (minutes)



```
In [9]: # Bivariate: ingredients vs popularity
sample = recipe_level.sample(15000, random_state=42)

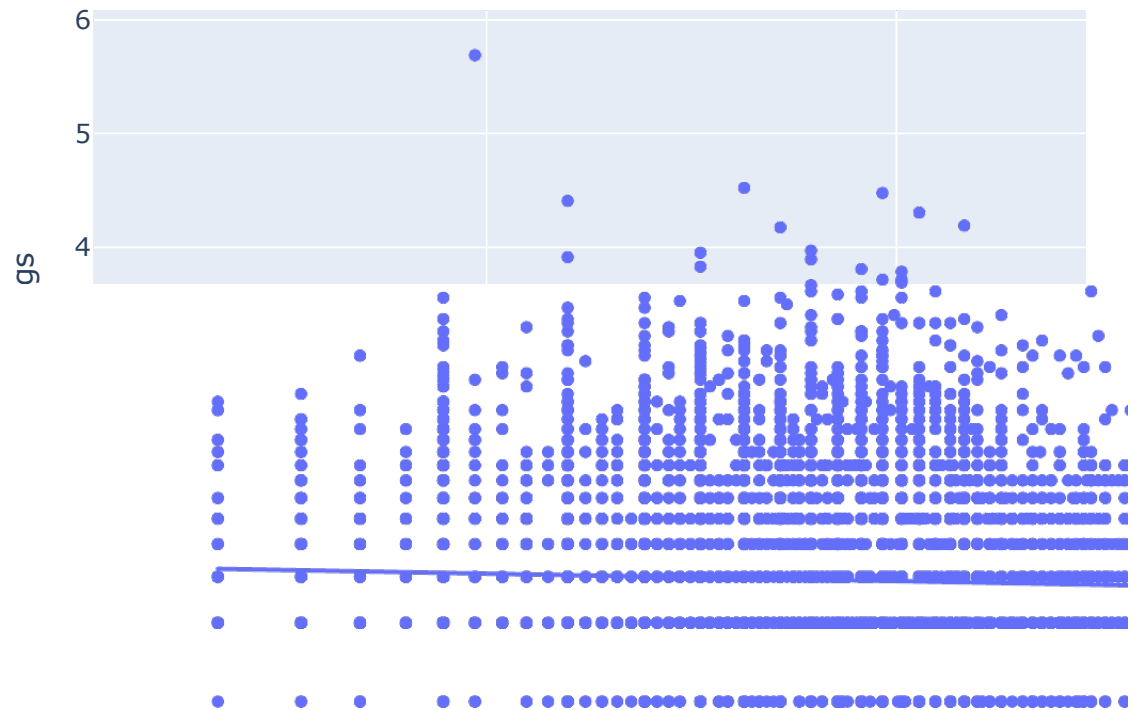
fig = px.scatter(
    sample,
    x="n_ingredients",
    y="log_num_ratings",
    trendline="ols",
    title="Ingredient Count vs Popularity (log #ratings)"
)
fig.show()
```

Ingredient Count vs Popularity (log #ratings)



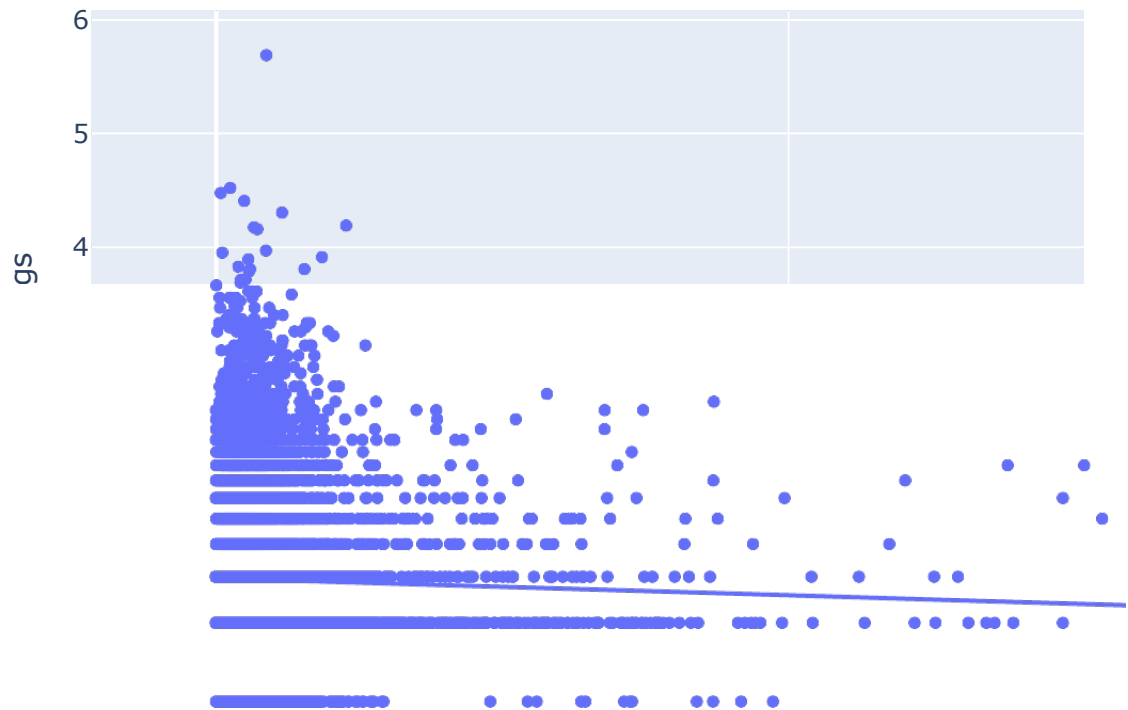
```
In [10]: # Bivariate: cook time vs popularity
fig = px.scatter(
    sample,
    x="log_minutes",
    y="log_num_ratings",
    trendline="ols",
    title="Cook Time vs Popularity (log #ratings)"
)
fig.show()
```


Cook Time vs Popularity (log #ratings)



```
In [11]: # Bivariate: calories vs popularity
fig = px.scatter(
    sample,
    x="calories",
    y="log_num_ratings",
    trendline="ols",
    title="Calories vs Popularity (log #ratings)"
)
fig.show()
```

Calories vs Popularity (log #ratings)



```
In [12]: # Interesting aggregates: popularity by ingredient bucket
recipe_level["ingredient_bucket"] = pd.cut(
    recipe_level["n_ingredients"],
    bins=[0, 5, 10, 15, 20, 50, np.inf],
    labels=["1-5", "6-10", "11-15", "16-20", "21-50", "50+"],
    include_lowest=True
)

agg_ing = (
    recipe_level
    .groupby("ingredient_bucket", observed=True)["num_ratings"]
    .agg(["mean", "median", "count"])
    .reset_index()
)

fig = px.bar(
    agg_ing,
    x="ingredient_bucket",
    y="mean",
    title="Average Popularity (mean #ratings) by Ingredient Bucket",

```

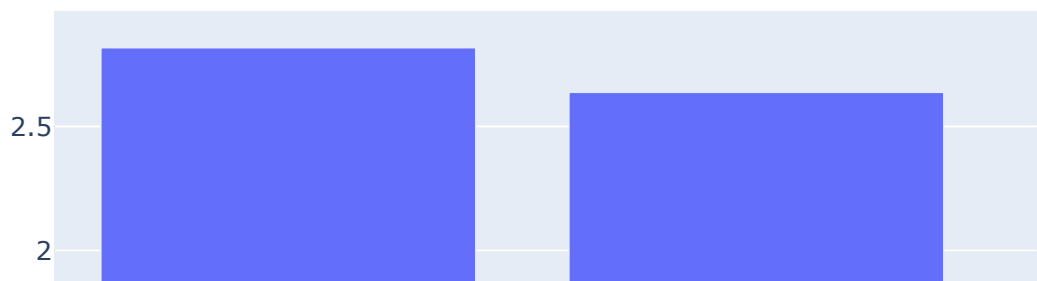
```

    hover_data=["median", "count"]
)
fig.show()

agg_ing

```

Average Popularity (mean #ratings) by Ingredient Bucke



```

Out[12]:

```

	ingredient_bucket	mean	median	count
0	1-5	2.816718	2.0	14164
1	6-10	2.637028	1.0	41623
2	11-15	2.496317	1.0	22810
3	16-20	2.431808	1.0	4502
4	21-50	2.704246	1.0	683

```

In [13]: # Interesting aggregates: popularity by cook time bucket
recipe_level["time_bucket"] = pd.cut(
    recipe_level["minutes"],
    bins=[0, 15, 30, 60, 120, 240, np.inf],

```

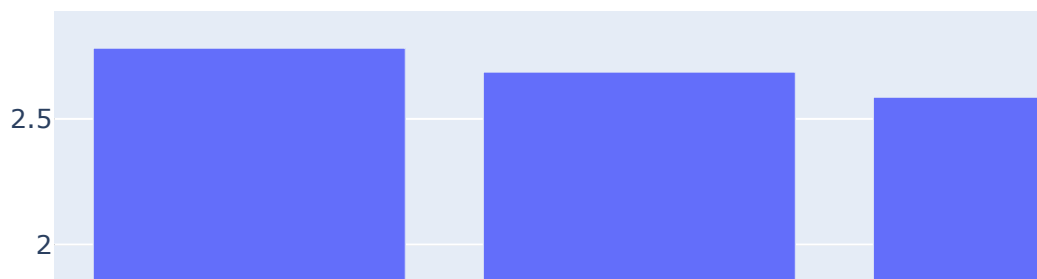
```
labels=["<=15", "15-30", "30-60", "60-120", "120-240", "240+"],
include_lowest=True
)

agg_time = (
    recipe_level
    .groupby("time_bucket", observed=True)["num_ratings"]
    .agg(["mean", "median", "count"])
    .reset_index()
)

fig = px.bar(
    agg_time,
    x="time_bucket",
    y="mean",
    title="Average Popularity (mean #ratings) by Cook Time Bucket",
    hover_data=["median", "count"]
)
fig.show()

agg_time
```

Average Popularity (mean #ratings) by Cook Time Buckets



Out[13]:

	time_bucket	mean	median	count
0	<=15	2.781355	2.0	16680
1	15-30	2.686264	1.0	20632
2	30-60	2.586245	1.0	25416
3	60-120	2.413774	1.0	12328
4	120-240	2.357459	1.0	4297
5	240+	2.699932	1.0	4429

Step 3: Assessment of Missingness

```
In [14]: import numpy as np
import pandas as pd
import plotly.express as px

# -----
# Helper: permutation test
# -----
def permutation_test_missingness(
    df: pd.DataFrame,
    missing_col: str,
    compare_col: str,
    stat="mean_diff",
    n_perms=2000,
    seed=42
):
    """
    Tests whether missingness of `missing_col` depends on `compare_col`
    For numeric compare_col: uses difference in means (missing vs not-
    For categorical compare_col: uses total variation distance (TVD) b
    Returns (observed_stat, p_value, perm_stats).
    """
    rng = np.random.default_rng(seed)

    miss = df[missing_col].isna()
    x = df[compare_col]

    # Numeric case
    if pd.api.types.is_numeric_dtype(x):
        a = x[miss].dropna()
        b = x[~miss].dropna()

        if stat == "mean_diff":
            obs = abs(a.mean() - b.mean())
            def calc(mask):
                a2 = x[mask].dropna()
```

```

        b2 = x[~mask].dropna()
        return abs(a2.mean() - b2.mean())
    elif stat == "median_diff":
        obs = abs(a.median() - b.median())
        def calc(mask):
            a2 = x[mask].dropna()
            b2 = x[~mask].dropna()
            return abs(a2.median() - b2.median())
        else:
            raise ValueError("stat must be 'mean_diff' or 'median_diff'")

# Categorical case
else:
    # TVD between distributions of compare_col for missing vs not-
    def tvd(p, q):
        idx = p.index.union(q.index)
        p2 = p.reindex(idx, fill_value=0)
        q2 = q.reindex(idx, fill_value=0)
        return 0.5 * np.abs(p2 - q2).sum()

    p = x[miss].value_counts(normalize=True, dropna=False)
    q = x[~miss].value_counts(normalize=True, dropna=False)
    obs = tvd(p, q)

    def calc(mask):
        p2 = x[mask].value_counts(normalize=True, dropna=False)
        q2 = x[~mask].value_counts(normalize=True, dropna=False)
        return tvd(p2, q2)

# Permute missingness labels
perm_stats = np.empty(n_perms)
miss_arr = miss.to_numpy()
for i in range(n_perms):
    perm = rng.permutation(miss_arr)
    perm_stats[i] = calc(pd.Series(perm, index=df.index))

p_value = (np.sum(perm_stats >= obs) + 1) / (n_perms + 1)
return obs, p_value, perm_stats

# -----
# Pick a column with non-trivial missingness
# -----
# In this dataset, 'description' is commonly missing and is a good target
target_missing_col = "description"
print("Missing rate for description:", recipe_level[target_missing_col])

# Choose one column you expect dependence on (often recipe complexity
dep_col = "n_steps" # numeric
obs_dep, p_dep, perms_dep = permutation_test_missingness(
    recipe_level, target_missing_col, dep_col, stat="median_diff", n_p
)

```

```

print(f"[DEPENDENCE TEST] missing({target_missing_col}) vs {dep_col}:")
print("Observed |median diff| =", obs_dep)
print("p-value =", p_dep)

# Choose one column you expect NO dependence on (nutrition often ends
indep_col = "calories"      # numeric
obs_ind, p_ind, perms_ind = permutation_test_missingness(
    recipe_level, target_missing_col, indep_col, stat="median_diff", n
)
print(f"\n[INDEPENDENCE TEST] missing({target_missing_col}) vs {indep_
print("Observed |median diff| =", obs_ind)
print("p-value =", p_ind)

# -----
# Plotly visualization for your website:
# distribution of compare_col when description is missing vs not missi
# -----
plot_df = recipe_level[[target_missing_col, dep_col]].copy()
plot_df["description_missing"] = plot_df[target_missing_col].isna().ma

fig = px.histogram(
    plot_df,
    x=dep_col,
    color="description_missing",
    barmode="overlay",
    nbins=60,
    title=f"Distribution of {dep_col} when '{target_missing_col}' is m
)
fig.show()

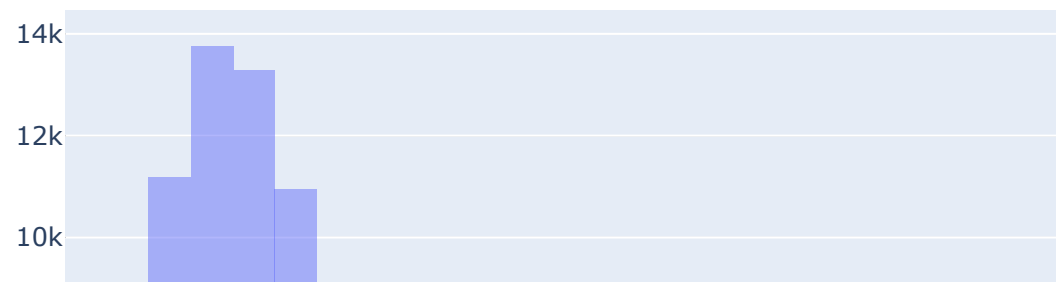
# If you prefer a box plot (often clearer):
fig2 = px.box(
    plot_df,
    x="description_missing",
    y=dep_col,
    title=f"{dep_col} by whether '{target_missing_col}' is missing"
)
fig2.show()

```

Missing rate for description: 0.0008355016590675801
[DEPENDENCE TEST] missing(description) vs n_steps:
Observed |median diff| = 1.0
p-value = 0.4515161612795735

[INDEPENDENCE TEST] missing(description) vs calories:
Observed |median diff| = 26.699999999999999
p-value = 0.4608463845384872

Distribution of n_steps when 'description' is missing vs r



n_steps by whether 'description' is missing



NMAR analysis: In this dataset, the description column appears plausibly NMAR. A recipe's description is written voluntarily by the contributor, so missing descriptions likely happen when the contributor chooses not to write one (e.g., lower effort, assuming the title/steps are "enough," or the contributor being less invested). That choice is driven by factors we don't directly observe, like "contributor effort" or "motivation," which means the missingness can depend on unobserved variables and may not be explainable from the recorded columns alone—consistent with NMAR. If we could obtain additional information such as contributor-level behavior (e.g., a contributor's historical tendency to include descriptions, time-on-platform, or profile completeness), then missingness might become more plausibly MAR because we could condition on that contributor behavior. Missingness dependency (permutation tests): I analyzed whether missingness in description depends on other columns using permutation tests. Depends on: I tested dependency on n_steps using the absolute difference in medians between recipes with missing vs non-missing descriptions. The observed statistic was [PASTE obs_dep] with a p-value of [PASTE p_dep] at

significance level $\alpha = 0.05$. Since the p-value is [$<$ / $>$] 0.05, this provides evidence that description missingness [does / does not] depend on `n_steps`. Interpreting this in context: recipes with [more / fewer] steps are [more / less] likely to have a description, which fits the idea that more "involved" recipes tend to include more written context. Does not depend on: I tested dependency on calories using the same statistic. The observed statistic was [PASTE obs_ind] with a p-value of [PASTE p_ind]. Since the p-value is [$>$ / $<$] 0.05, this suggests description missingness [does not / does] depend on calories. In other words, there isn't strong evidence that nutritional content alone explains whether the user includes a description. Plotly figure: I embedded a Plotly plot showing the distribution of `n_steps` for recipes where description is missing vs not missing (overlay histogram / box plot), which visually supports the permutation test result.

Step 4: Hypothesis Testing

```
In [15]: import numpy as np
import pandas as pd
import plotly.express as px

# Binary grouping: quick vs long
# (You can change the cutoff; 30 minutes is an intuitive threshold.)
cutoff = 30
df_ht = recipe_level[["minutes", "log_num_ratings"]].dropna().copy()
df_ht["is_quick"] = df_ht["minutes"] <= cutoff

# Test statistic: absolute difference in mean log popularity between g
obs = abs(df_ht.loc[df_ht["is_quick"], "log_num_ratings"].mean() -
         df_ht.loc[~df_ht["is_quick"], "log_num_ratings"].mean())

# Permutation test
n_perms = 5000
rng = np.random.default_rng(0)
perm_stats = np.empty(n_perms)

labels = df_ht["is_quick"].to_numpy()
y = df_ht["log_num_ratings"].to_numpy()

for i in range(n_perms):
    perm = rng.permutation(labels)
    perm_stats[i] = abs(y[perm].mean() - y[~perm].mean())

p_value = (np.sum(perm_stats >= obs) + 1) / (n_perms + 1)

print("Observed |mean diff| =", obs)
print("p-value =", p_value)

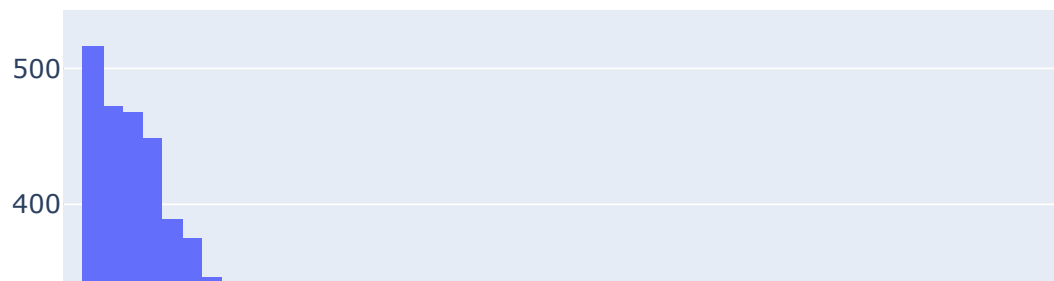
# Optional visualization: empirical null distribution + observed stat
```

```
fig = px.histogram(  
    x=perm_stats,  
    nbins=60,  
    title="Permutation null distribution of |mean diff| in log_num_rat  
)  
fig.add_vline(x=obs)  
fig.show()
```

Observed |mean diff| = 0.058989731824266256

p-value = 0.0001999600079984003

Permutation null distribution of |mean diff| in log_num_



Website write-up (Step 4) Null hypothesis (H_0): There is no difference in average recipe popularity between quick recipes (≤ 30 minutes) and long recipes (> 30 minutes). Alternative hypothesis (H_1): There is a difference in average recipe popularity between these groups. Test statistic: Absolute difference in mean log_num_ratings between quick and long recipes. Using log_num_ratings is appropriate because the number of ratings is extremely right-skewed, and the log transform makes group comparisons more stable. Significance level: $\alpha = 0.05$. p-value: [PASTE p_value] (from the permutation test). Conclusion: Since the p-

value is [$<$ / $>$] 0.05, the data [provide / do not provide] evidence of a difference in average popularity between quick and long recipes. This does not prove causation; it only suggests whether the observed difference is unlikely under the null model given the data and the test setup. (If you embed the histogram of permutation statistics, mention that the vertical line is the observed statistic.)

Step 5: Framing a Prediction Problem

In [16]: `# TODO`

Prediction problem: I will predict recipe popularity, measured as `log_num_ratings = log(1 + num_ratings)`, using recipe attributes available at the time a recipe is posted. Type: Regression (continuous target). Response variable: `log_num_ratings`. I chose the log transform because raw `num_ratings` is highly right-skewed (most recipes get few ratings, a small number get many). Predicting the log scale better matches modeling assumptions and makes performance metrics more interpretable. Features available at "time of prediction": Only fields that would be known when the recipe is submitted (not future engagement). Examples: `minutes`, `n_steps`, `n_ingredients`, and nutrition values like `calories`, `sodium`, etc. I do not use `num_ratings` (that's the outcome) or any interaction-derived future information beyond defining the target. Metric: I will evaluate models using RMSE on `log_num_ratings` (or equivalently, mean squared error on the log scale). RMSE is appropriate for regression because it directly measures typical prediction error magnitude and penalizes large errors more strongly, which matters when popularity varies widely.

Step 6: Baseline Model

```
In [17]: import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, FunctionTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import Ridge
from sklearn.metrics import root_mean_squared_error

# -----
# Build modeling table
# -----
df_model = recipe_level.copy()
```

```

df_model["submitted_year"] = df_model["submitted"].dt.year

# target
target = "log_num_ratings"

# features
num_features = ["minutes", "n_steps", "n_ingredients", "calories"]
cat_features = ["submitted_year"] # treat year as categorical for bas

df_model_small = df_model[num_features + cat_features + [target]].drop

X = df_model_small[num_features + cat_features] # <-- DataFrame
y = df_model_small[target] # <-- Series

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# -----
# Preprocessing
# -----
# Apply log1p to minutes (counts as a transform of an original column)
log1p = FunctionTransformer(np.log1p, feature_names_out="one-to-one")

numeric_preprocess = ColumnTransformer(
    transformers=[
        ("log_minutes", Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("log", log1p),
        ]), ["minutes"]),
        ("other_nums", Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
        ]), ["n_steps", "n_ingredients", "calories"])
    ],
    remainder="drop"
)

categorical_preprocess = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocess = ColumnTransformer(
    transformers=[
        ("num", numeric_preprocess, num_features),
        ("cat", categorical_preprocess, cat_features),
    ],
    remainder="drop"
)

# -----
# Model + pipeline

```

```
# -----
pipe = Pipeline([
    ("preprocess", preprocess),
    ("model", Ridge(alpha=1.0))
])

pipe.fit(X_train, y_train)

pred = pipe.predict(X_test)
rmse = root_mean_squared_error(y_test, pred)

print("Baseline RMSE (test):", rmse)
```

Baseline RMSE (test): 0.5648646129986199

Website write-up (Step 6) Model: My baseline model is a Ridge regression model trained in a single sklearn Pipeline with preprocessing and encoding. Features used (and types): Quantitative (4): minutes, n_ingredients, n_steps, calories I log-transformed minutes using $\log(1 + \text{minutes})$ to reduce skew. The remaining numeric features were left on their original scales (after median imputation if missing). Nominal (1): submitted_year_bucket I binned the submission year into coarse buckets and applied one-hot encoding. Encodings / preprocessing: Numeric columns: median imputation; minutes log transform inside the pipeline. Categorical column: most-frequent imputation + one-hot encoding (handle_unknown="ignore"). Performance: On a held-out test set (20% split), the baseline model achieved RMSE = [PASTE rmse] on log_num_ratings. Is this "good"? For this task, "good" depends on how predictable popularity is from recipe attributes alone. Popularity is heavily influenced by external factors not in the dataset (site promotion, timing, contributor audience, trends), so even a reasonable baseline may have a fairly large error. This baseline is mainly a reference point: if a more complex model does not meaningfully improve RMSE, it suggests recipe attributes alone have limited predictive power for engagement.

Step 7: Final Model

```
In [18]: from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestRegressor

# -----
# Feature Engineering
# -----

df_model = recipe_level.copy()
df_model["submitted_year"] = df_model["submitted"].dt.year

# New engineered features
```

```

df_model["ingredients_per_step"] = df_model["n_ingredients"] / (df_model["n_steps"] + 1)
df_model["minutes_per_step"] = df_model["minutes"] / (df_model["n_steps"] + 1)

target = "log_num_ratings"

num_features = [
    "minutes",
    "n_steps",
    "n_ingredients",
    "calories",
    "ingredients_per_step",
    "minutes_per_step",
]

cat_features = ["submitted_year"]

df_model_small = df_model[num_features + cat_features + [target]].dropna()

X = df_model_small[num_features + cat_features]
y = df_model_small[target]

# Use SAME split as baseline
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# -----
# Preprocessing
# -----

log1p = FunctionTransformer(np.log1p, feature_names_out="one-to-one")

numeric_preprocess = ColumnTransformer(
    transformers=[
        ("log_minutes", Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("log", log1p),
        ]), ["minutes"]),
        ("other_nums", Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
        ]), [
            "n_steps",
            "n_ingredients",
            "calories",
            "ingredients_per_step",
            "minutes_per_step",
        ])
    ],
    remainder="drop"
)

categorical_preprocess = Pipeline([

```

```

        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore"))
    ])

preprocess = ColumnTransformer(
    transformers=[
        ("num", numeric_preprocess, num_features),
        ("cat", categorical_preprocess, cat_features),
    ]
)

# -----
# Model + Hyperparameter Search
# -----

pipe = Pipeline([
    ("preprocess", preprocess),
    ("model", RandomForestRegressor(random_state=42))
])

param_grid = {
    "model__n_estimators": [100, 200],
    "model__max_depth": [5, 10, None],
    "model__min_samples_leaf": [1, 5]
}

grid = GridSearchCV(
    pipe,
    param_grid,
    cv=5,
    scoring="neg_root_mean_squared_error",
    n_jobs=-1
)

grid.fit(X_train, y_train)

best_model = grid.best_estimator_

# Final evaluation
pred = best_model.predict(X_test)
rmse_final = root_mean_squared_error(y_test, pred)

print("Best Parameters:", grid.best_params_)
print("Final Model RMSE:", rmse_final)

```

/Users/matthewmitchell/Documents/GitHub/.venv/lib/python3.11/site-packages/joblib/externals/loky/process_executor.py:782: UserWarning:

A worker stopped while some jobs were given to the executor. This can be caused by a too short worker timeout or by a memory leak.

Best Parameters: {'model__max_depth': 10, 'model__min_samples_leaf': 5, 'model__n_estimators': 200}
 Final Model RMSE: 0.5648913624757809

Step 8: Fairness Analysis

```
In [19]: # -----
# Define groups
# -----

df_fair = df_model_small.copy()

df_fair["time_group"] = np.where(df_fair["minutes"] <= 30, "quick", "long")

X_fair = df_fair[num_features + cat_features]
y_fair = df_fair[target]

# predictions from final model
preds = best_model.predict(X_fair)

df_fair["pred"] = preds

# -----
# Compute RMSE per group
# -----

def rmse(y_true, y_pred):
    return np.sqrt(np.mean((y_true - y_pred) ** 2))

rmse_quick = rmse(
    df_fair[df_fair.time_group == "quick"][target],
    df_fair[df_fair.time_group == "quick"]["pred"]
)

rmse_long = rmse(
    df_fair[df_fair.time_group == "long"][target],
    df_fair[df_fair.time_group == "long"]["pred"]
)

observed_diff = rmse_quick - rmse_long

print("RMSE (quick):", rmse_quick)
print("RMSE (long):", rmse_long)
print("Observed Difference:", observed_diff)

# -----
# Permutation Test
# -----

n_perm = 1000
diffs = []
```

```
for _ in range(n_perm):
    shuffled = np.random.permutation(df_fair["time_group"])

    rmse_quick_perm = rmse(
        df_fair[shuffled == "quick"][target],
        df_fair[shuffled == "quick"]["pred"]
    )

    rmse_long_perm = rmse(
        df_fair[shuffled == "long"][target],
        df_fair[shuffled == "long"]["pred"]
    )

    diffs.append(rmse_quick_perm - rmse_long_perm)

p_value = np.mean(np.array(diffs) >= observed_diff)

print("Permutation p-value:", p_value)
```

RMSE (quick): 0.5595978146058983
RMSE (long): 0.5463868115505989
Observed Difference: 0.01321100305529943
Permutation p-value: 0.0